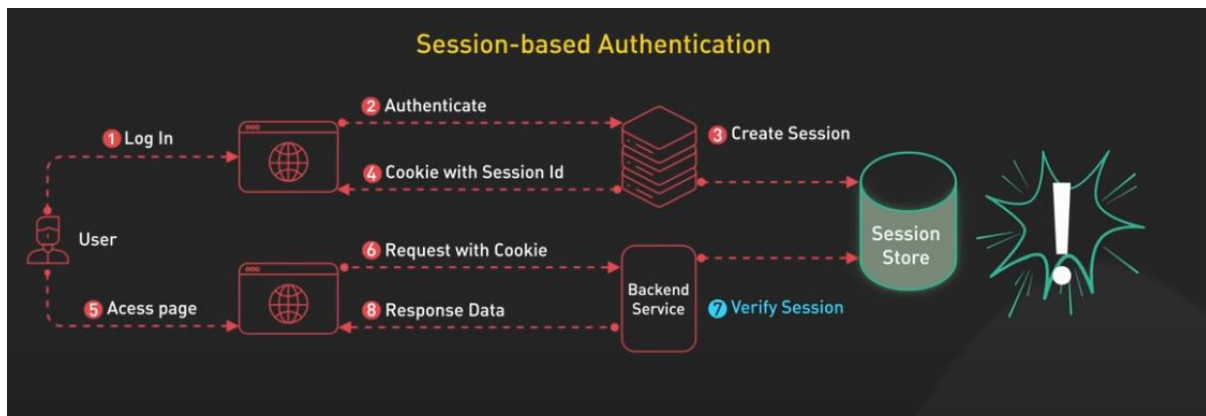


1. Session-Based Authentication



- 사용자가 로그인 요청을 하면 서버는 사용자의 인증을 처리하기 위해 세션을 생성 이때 사용자 구분을 위한 session id값이 발급되고 이 값을 쿠키에 전달(1~3)
- 세션은 서버상 정보를 저장하기 위한 강력한 Attribute로서 주로 DB에 입력된 중요한 사용자 정보를 저장하는 수단이다.
- 쿠키는 응답 시 클라이언트로 전송되고 다음 번 서버에 요청 시 다시 서버에 전달된다.(6) 이때 서버는 클라이언트 인증을 위한 수단으로 쿠키안에 있는 session id값을 꺼내 확인 하고 이 과정을 Verify Session 이라 한다.(7)

➔ 중요한 점은 세션 기반 인증을 사용하면 서버가 세션 생성 및 데이터 저장을 담당한다는 것이다.



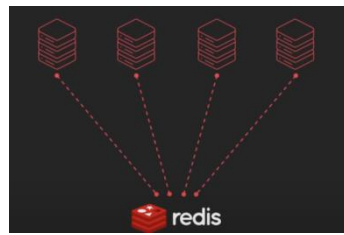
장점

- 세션 취소가 간단하다.
 - 세션이 서버에 저장됨으로 언제든지 삭제 혹은 무효화 시킬 수 있다.

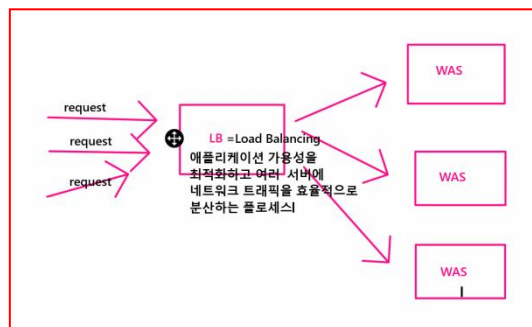
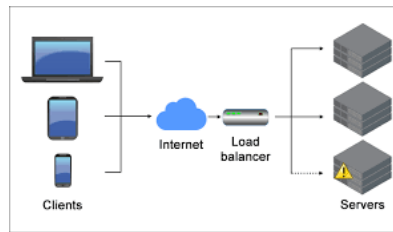
단점

- 어플리케이션이 여러 서버에서 실행되는 분산 시스템에서는 모든 서버가 동일한 세션 데

이타에 액세스해야 한다.



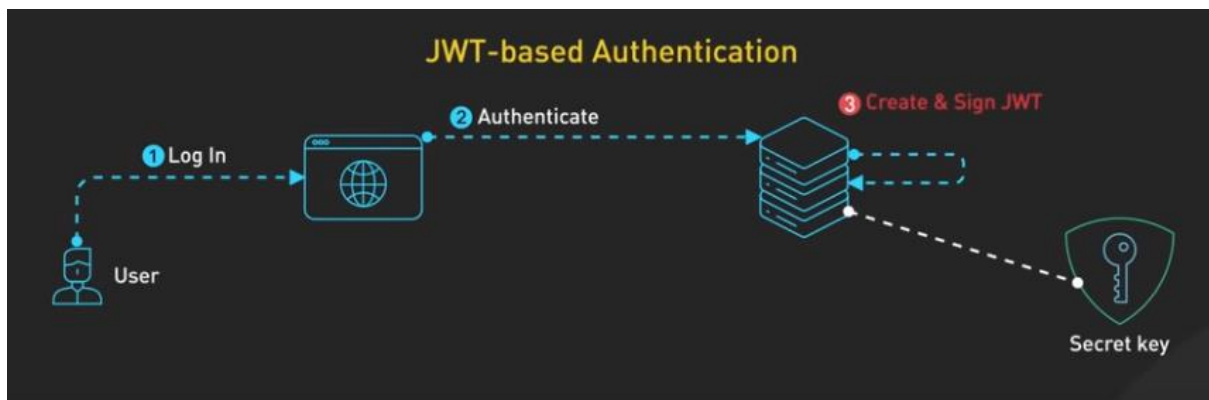
VS



이러한,

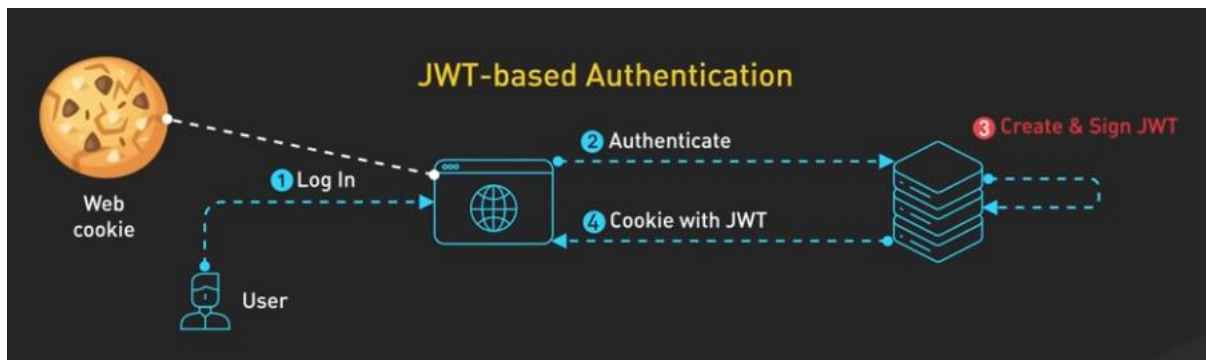
Session의 단점을 보완하기 위해 탄생한 방식이 JWT 방식이다.

2. JWT-Based Authentication

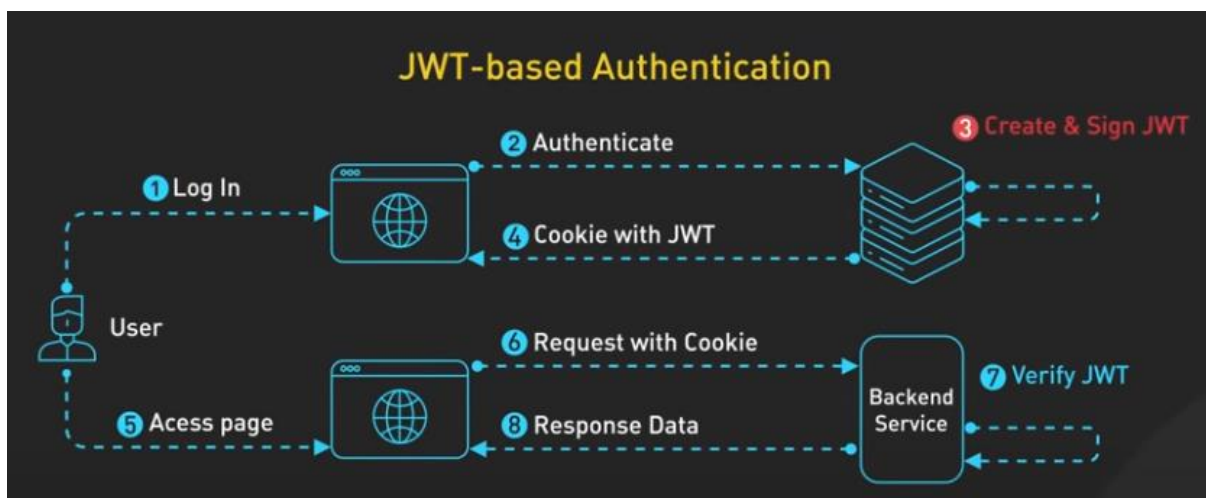


- 사용자 요청하면 서버는 사용자의 인증을 확인하기 위해서 JWT생성 (1,2,3)
- 우리는 LoginFilter 클래스에서 성공시 호출되는 successAuthentication() 함수에서 JWT토큰 생성을 했다. 이때 사용자정보, 권한 만료시간으로 토큰을 생성했다. 사용자 정보에는 이름, 아이디, 권한만을 등록하고 패스워드 정보는 저장되지 않도록 했다.
- 그리고 서버는 비밀키로 JWT에 서명을 한다. signWith(secretKey)

이 서명은 토큰의 무결성을 보장하기 위함이며 변조를 방지해준다.



- 서버는 응답 시 Header 부분에 JWT토큰 정보를 실어서 클라이언트로 보낸다. (4)
- 클라이언트는 일반적으로 LocalStorage나 Cookie에 JWT토큰을 저장한다.
- Access Token 사용시 주로 LocalStorage 저장한다. XSS 공격에 취약한 단점을 가진다.
- Refresh Token 사용시 주로 Cookie 저장한다. CSRF 공격에 대비해야 한다.



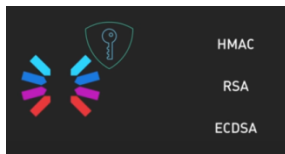
- 다음 번 요청 시(5) 클라이언트는 Request 헤더에 JWT를 실어서 서버로 전송된다(6)
- 서버는 JWT서버를 확인한다(7). 서명이 유효하다면 토큰의 데이터를 신뢰하고 이를 인증 및 승인 요청 로직에 사용한다.

➔ 중요한 점은 JWT를 사용하면 서버가 세션 상태를 저장하지 않는다.

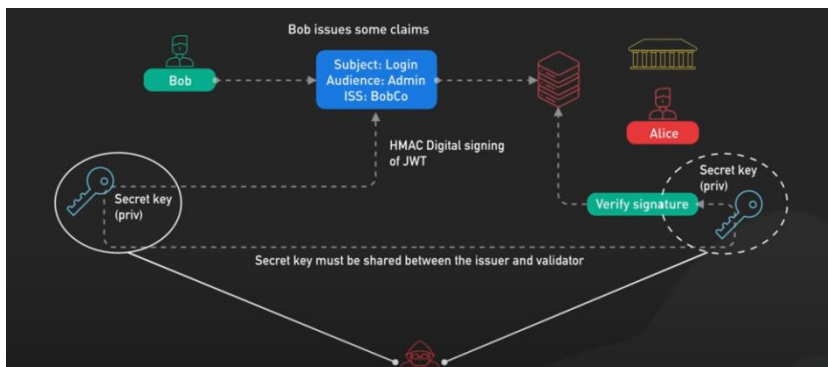
필요한 모든 데이터는 클라이언트 토큰 안에 포함된다.

이로 인해 Stateless상태가 보장된다.

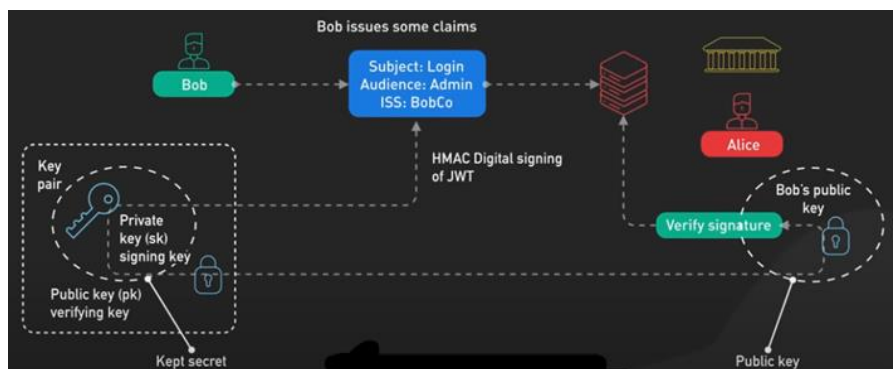
3. JWT 서명에 사용되는 알고리즘



우리가 사용하는 HS256 알고리즘은 HMAC + RSA 결합방식

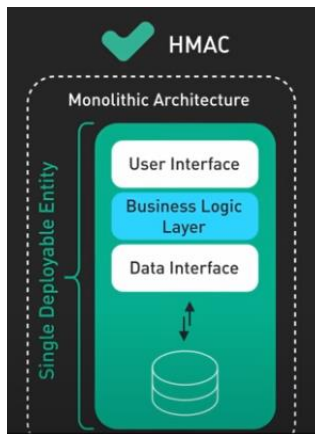


HMAC는 대칭형 서명방법으로 동일한 비밀키가 토큰 서명 및 확인에 사용된다. 간단하고 효율적인 방법이지만 토큰을 확인해야 하는 모든 서비스에서 비밀키를 공유해야 함으로 보안문제가 발생할 수 있다.



반면, RSA, ECDSA는 비대칭 서명방법이다. 개인키를 사용해서 토큰에 서명하고 공개키를 사용하여 토큰을 확인한다. 이를 통해 개인키가 비밀로 유지되고 서명에만 사용되는 동시에 서비스에서는 공개키를 사용해서 토큰을 확인할 수 있는 보다 안전한 아키텍처가 가능해졌다. HMAC에 비해 약간의 복잡성과 계산 오버헤드가 추가되었다.

서명 알고리즘 선택은 보안 요구사항 및 시스템 아키텍처에 따라 선택이 달라진다.



모놀리식 어플리케이션이거나 시스템의 모든 서비스를 신뢰할 수 있는 경우 HMAC로 충분할 수 있다. 하지만 마이크로서비스 아키텍처가 있거나 JWT를 신뢰할 수 없는 타사 서비스와 공유해야 하는 경우 RSA 또는 ECDSA가 더 안전한 솔루션을 제공한다.

4. JWT 토큰 만료 처리하기

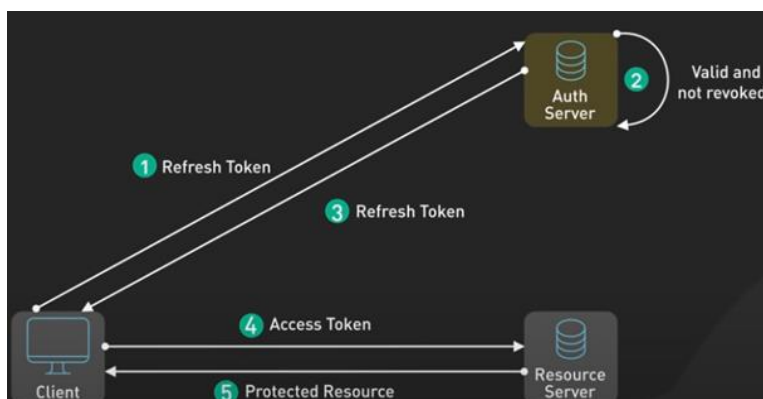


토큰을 탈취당한 경우 만료시까지 사용할 수 있기 때문에 이 문제는 중요하다. 이를 위해서 Access Token과 Refresh Token을 함께 사용할 수 있다.

- Access Token 은 각 요청의 인증에 사용되는 실제 JWT이다. 만료시간은 일반적으로 15분 정도로 짧게 잡는다.
- Refresh Token은 만료시간이 훨씬 길어 며칠 혹은 몇 주 정도 지정한다. DB에 저장된다. Access Token 을재발급 할 때 필요한 토큰이다.

사용자는 Access Token이 수명을 다하면 Refresh Token을 보낸다.

서버는 DB에 저장되어 있는 값과 비교해보고 맞으면 새로운 Access Token을 발급해준다.



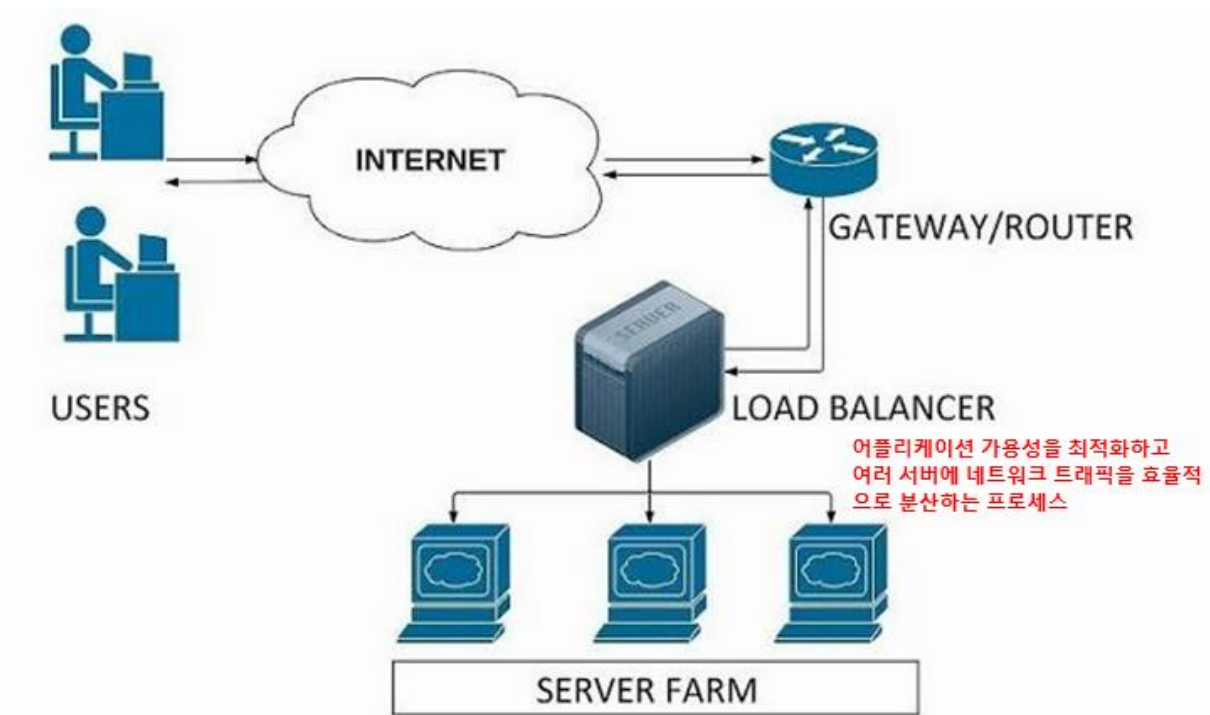
- Access Token이 만료되면 사용자에게 다시 로그인 하도록 요구하는 대신 클라이언트는 Refresh Token을 서버의 특수 토큰 EndPoint로 보낼 수 있다.
서버는 Refresh Token이 유효한지 확인한다(1,2,3)
- 모든 것이 확인되면 서버는 새 Access Token을 발급한다.(4,5)
이 프로세스는 사용자의 요청,응답 상호작용 없이 백그라운드에서 진행된다.



Access Token은 만료시간을 짧게 해서 토큰 도난시 잠재적인 오용 가능성을 제한하는 한편, **Refresh Token**은 사용자가 반복적으로 로그인을 계속 할 필요 없이 장기간 인증 상태를 유지할 수 있도록 한다.

단 Refresh Token은 Access Token이 만료된 경우에만 필요한 토큰으로 실제 인증요청 처리에는 사용되지 않는다.

5. 세션 인증 기반에서의 LoadBalance 문제점 해결책과 세션 클러스터링



그렇다면,

JWT가 Session을 대체하기에 완벽할까?

모든 사용자의 정보를 서버가 기억하고 있다는 것은 큰 이점이다.

예를 들어, 한 기기에만 로그인 가능한 서비스라고 할 때, Session기술은 pc에서 로그인한 상태의 어떤 사용자가 핸드폰에서 또 로그인하면 서버에서 PC상태 로그인을 로그아웃을 시켜 기존 세션을 종료할수 있다.

어떤 토큰이 해커에게 탈취당한 경우 해커가 가져가버린 토큰을 무효화 할수 있는 방법이 없다. 때문에 실제 서비스 중에 JWT만으로 인가를 구현하는 곳은 많지 않다.

그래서,

탈취 당했을 때를 방어하기 위해 로그인 만료 시간을 짧게 설정하는 방법을 사용한다.

그러나,

만료시간이 짧으면 사용자들이 잦은 로그인을 해야하는 번거로움이 생긴다.

이를 보완하기 위해서 쓰는 방법으로 로그인하면 2개의 토큰을 발급한다.

Access Token : 만료시간을 짧게(매번 인가를 받을 때 마다 필요한 토큰으로 수명시간이 짧은 토큰)

Refresh token : 만료시간을 길게 만들고 DB에 저장한다.

Access Token을 재발급 할 때 필요한 토큰이다.

사용자는 access token이 수명을 다하면 refresh token을 보낸다.

서버는 DB에 저장되어 있는 값과 비교해보고 맞으면 새로운 access token을 발급해준다.

이렇게 해서 중간에 액세스 토큰을 탈취당해도 오래 쓰지는 못한다.

누구를 강제 로그아웃 시키려면 refresh token을 DB에서 지워버리고 토큰 갱신이 안되게 하면 된다.

하지만,

이것 또한 짧게나마 탈취당해 살아 있는 액세스 토큰을 바로 차단 할 방법은 없다. 그냥 토큰 하나로 쓰는 것보다는 좋지만 이것이 완전한 해결책은 안 된다.

이러한 장단점이 있기에 이를 적용하기에 내 서비스가 적합한지를 충분히 고려해서 사용해야한다.

